Bases de la programmation

Durée : 1 h 10 — Barème : 20 points

1- Seules les constructions de base de Python vues en cours sont autorisées (variables, affectations, tests, boucles, listes, indices, fonctions et `return`). Les fonctionnalités donnant directement le résultat (par exemple `min`, `max`, `sum`, `sort`, `sorted`, `count`, `index`, etc.) sont interdites. La fonction `len` est autorisée.

2- Aucun document autorisé.

3- Les téléphones et autres appareils connectés doivent être mis dans les sacs fermés et surtout pas dans les poches.

Exercice 1 — Compréhension d'un programme (5 points)

```
1  def calcul(valeurs, x, y):
2      s = 0
3      n = 0
4      i = 0
5      while i < len(valeurs):
6          if valeurs[i] >= x and valeurs[i] <= y:
7              s = s + valeurs[i]
8              n = n + 1
9          i = i + 1
10     if n == 0:
11         return 0
12     return s / n
13
14 L = [4, 13, 7, 16, 3, 10]
15 r = calcul(L, 8, 13)
16 print(r)
```

1.1 Qu'affiche le programme ci-dessus ? **(1 point)**

1.2 Expliquer en une phrase ce que fait la fonction `calcul`. Répondre sous la forme :

« La fonction prend en arguments … et renvoie … ». **(3 points)**

1.3 Expliquer pourquoi le test « `if n == 0: return 0` » est nécessaire. **(1 point)**

Exercice 2 — Minimum et maximum d'une liste (5 points)

2.1 Écrire une fonction **`min_max(L)`** qui prend en argument une liste non vide `L` et renvoie, dans cet ordre : la plus petite valeur, son indice, la plus grande valeur et son indice. Si le minimum ou le maximum apparaît plusieurs fois, renvoyer l'indice de sa première occurrence. **(4 points)**

Exemple : pour `L = [1, 2, 1, 1, 2]`, la fonction doit renvoyer `1, 0, 2, 1`.

2.2 Écrire un programme permettant de tester la fonction **`min_max`**. **(1 point)**

Exercice 3 — Recherche et suppression des doublons (5 points)

3.1 Écrire une fonction **`present(L, x)`** qui renvoie `True` si `x` est présent dans `L`, et `False` sinon. **(2 points)**

3.2 Écrire une fonction **`sans_doublon(L)`** qui renvoie une nouvelle liste contenant les éléments de `L` sans doublon, dans leur ordre d'apparition. Elle devra obligatoirement utiliser la fonction **`present`**. **(2 points)**

Exemple : `[2, 5, 2, 7, 5, 3, 2]` donne `[2, 5, 7, 3]`.

3.3 Écrire un programme permettant de tester **`sans_doublon`**. **(1 point)**

Consignes : l'opérateur `in` est interdit dans un test `if`. La liste `L` ne doit pas être modifiée.

Exercice 4 — Description des répétitions successives (5 points)

Écrire une fonction **`repetitions(L)`** qui prend en argument une liste non vide `L` et renvoie une nouvelle liste contenant, pour chaque suite de valeurs identiques consécutives, une liste de deux éléments : la valeur et son nombre de répétitions successives. **(5 points)**

Exemple : pour `L = [2, 2, 2, 5, 5, 7, 3, 3, 3, 3]`, la fonction doit renvoyer `[[2, 3], [5, 2], [7, 1], [3, 4]]`.
Ainsi, `[2, 3]` signifie que la valeur 2 apparaît 3 fois consécutivement.